Employment & Economic Trends: Dashboard

TABLE OF CONTENTS

Introduction and Goal

Building Line Plots

Building Choropleth

Use Cases and Demo

THE GOAL

- Design a dashboard using FRED data to provide an overview of employment statistics by industry.

- Use to identify potential events and to provide statistical background to identified events.

- Three Components:
  - Employee headcounts by industry over time (Line)
  - Employment by industry by state over time (Choropleth)
  - Wage by industry over time (Line)

A Comparison to Optimization

"...Formulate and solve an optimization model for monthly production in a factory given a set of products, work hours, and demand..."

1. **Purpose:** Determine optimal production
2. **Scope:** Within the constraints of predetermined variables
3. **Content:** An answer that is correct and explainable

***In this case there is a correct answer that everyone should arrive at.*

THE DASHBOARD's PURPOSE

Before any work could begin, we needed a purpose for the dashboard.

DECISION:

1. Center the dashboard around economic research.
2. Identify trends that warrant further research.
3. See the impact of events on statistics.

DASHBOARD CONTENT

Address the scope using techniques from our course while pushing our understanding of said techniques.

Employee Headcounts by Industry Over Time

Time series with a secondary axis overlay option to show a recession indicator.

Employee Headcounts by Industry-State Over Time

Choropleth to show presence of an industry in each state.

Average Hourly Wage by Industry Over Time

Time series with a secondary axis overlay option to show CPI.

BUILDING LINE PLOTS (BASIC INPUTS)

- The headcount and wage graphs operate on the same code
- Build data frame outside of callback using FRED datasets for each industry
- Incorporate a dcc checkbox to take user input
- Within callback, filter data frame by selected industries and plot.

```
for i in range(len(series_identifiers)):

    params = {
    'series_id': series_identifiers[i],
    'api_key': fred_api_key,
    'file_type':'json'
    }

    response = requests.get(url, params = params)
    if response.status_code == 200:
        data = response.json()
        obs = data.get('observations', [])
        dftemp = pd.DataFrame(obs)
        dftemp['id'] = series_labels[i]
        dftemp['value'] = pd.to_numeric(dftemp['value'], errors = 'coerce')
        dftemp['date'] = pd.to_datetime(dftemp['date'], errors = 'coerce')
        frame = [df, dftemp]
        df = pd.concat(frame)
```

```
filtered_df = df[df['id'].isin(industries)]
```

BUILDING LINE PLOTS (DATE)

- A user may want to see a specific range with more clarity
- Incorporate a dcc date range
- Within callback, use the start date and end date to filter the data frame and plot

```
filtered_df = filtered_df[filtered_df['date']<= end]
filtered_df = filtered_df[filtered_df['date']>= start]
```

BUILDING LINE PLOTS (TWO Y-AXES)

- Plotly does not natively support two y-axes
- Required plotly subplots and plotly graph objects
- Include a separate checklist element to act as a toggle
- Toggle determines how you build graph

```
if 'recess' in toggle:
    ids = filtered_df['id'].unique()
    fig = make_subplots(specs = [[{'secondary_y': True}]])
    for i in ids:
        df_temp = filtered_df[filtered_df['id']== i ]
        fig.add_trace(
        go.Scatter(x = df_temp['date'], y = df_temp['value'], name = i, mode = 'lines'),
        secondary_y= False,
        )
    fig.add_trace(
    go.Scatter(x = filtered_dfrecess['date'], y = filtered_dfrecess['value'], name = 'GDP Rece
    secondary_y= True,
    )
    fig.update_layout(xaxis_title = 'Year', yaxis_title = 'Employed Persons in Thousands')
    fig.update_yaxes(title_text = 'GDP Recession Indicator(%)', secondary_y = True)
    fig.update_yaxes(range = [0, None], secondary_y= False)
else:
    fig = px.line(filtered_df, x = 'date', y= 'value', color = 'id',
                  labels = {
                      'value': 'Employed Persons in Thousands',
                      'date':'Year',
                      'id': 'Industry'
                  })
    fig.update_yaxes(range = [0, None])

return fig
```

BUILDING CHOROPLETH

- Required industry headcounts by both state and industry.
  - Massive dataset, requires external saving
- Year and month selector included to be able to see every release
- Some states are missing data depending on the industry
- Some industries are so concentrated in some states that it makes the choropleth hard to read

```
def fetch_data(industry_key):
    df = pd.read_csv("FinalDF.csv")
    df = df.dropna(subset=['date', 'value_empindustry', 'value_nonfarmemp', 'id', 'state'])

    df['date'] = pd.to_datetime(df['date'], errors='coerce')
    df['value_empindustry'] = pd.to_numeric(df['value_empindustry'], errors='coerce')
    df['value_nonfarmemp'] = pd.to_numeric(df['value_nonfarmemp'], errors='coerce')
```

BUILDING CHOROPLETH (ADJUSTMENT)

- Adjusted Choropleth to show industry presence as a percentage of total non-farm jobs.
- Ensures that more populous states are not overrepresented on Choropleth

```
df['value'] = df['value_empindustry'] / df['value_nonfarmemp'] * 100
```

USE CASES

Finding the statistical impact of an event on employment and wages in a given industry:

- How large were the COVID-19 employment dips for each industry?
- How have wages in industries responded to inflation post-COVID?

Identifying trends and concentrations for further research:

- Why are education and health services resistant to recessions?
- Why does Texas have a high concentration of mining and logging industries?